

实验 2 用 PID 实施进程控制 背景知识

Linux 系统的命令行、前台作业和后台作业

同济大学计算机系操作系统荣誉班 邓蓉

一、用管道连起来的是一个作业

Linux 系统的作业需要执行多个应用程序时，用管道 “|” 把它们连起来。前一个程序的标准输出（printf 的东西，或者是 cout 的东西）是后一个程序的标准输入（就，后一个程序 scanf 或 cin，读前一个程序的输出）

图中是一个作业。Linux 系统中，用管道相连的 4 个程序组成一个作业。

[illegible]

图 1

接纳作业后，系统创建 4 个进程 10920，10921，10922，10923，执行作业中出现的 4 个程序。

这是一个前台作业。接收终端输入，可以优先向屏幕输出。

这个作业由 4 个进程合作完成，前面一个进程的标准输出连至后一个进程的标准输入。。。故，只有第四个进程才可以写屏幕。所以，屏幕上全是 D。

在上述命令行的最后，加 `&` 。这是一个后台作业。系统立即创建 4 个进程来执行这个作业。

二、“;”用来分割一个命令行里出现的多个作业

这个命令行，4 个作业。

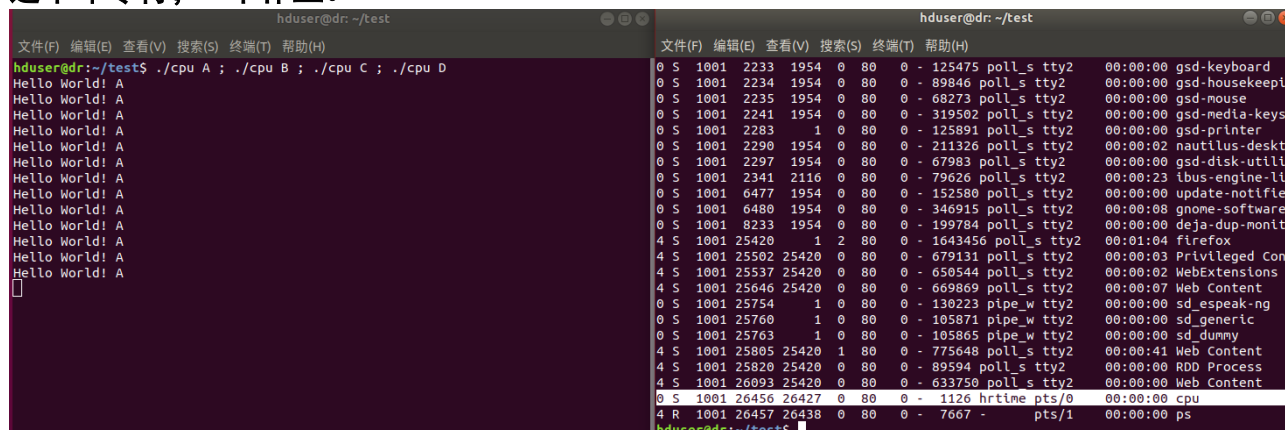


图 2

每个终端，只能有一个前台作业！ 否则，这个终端的标准输入输出就乱套啦。
所以，目前只接纳一个作业，执行程序 ./cpu A。看系统只派发了一个 cpu 进程。

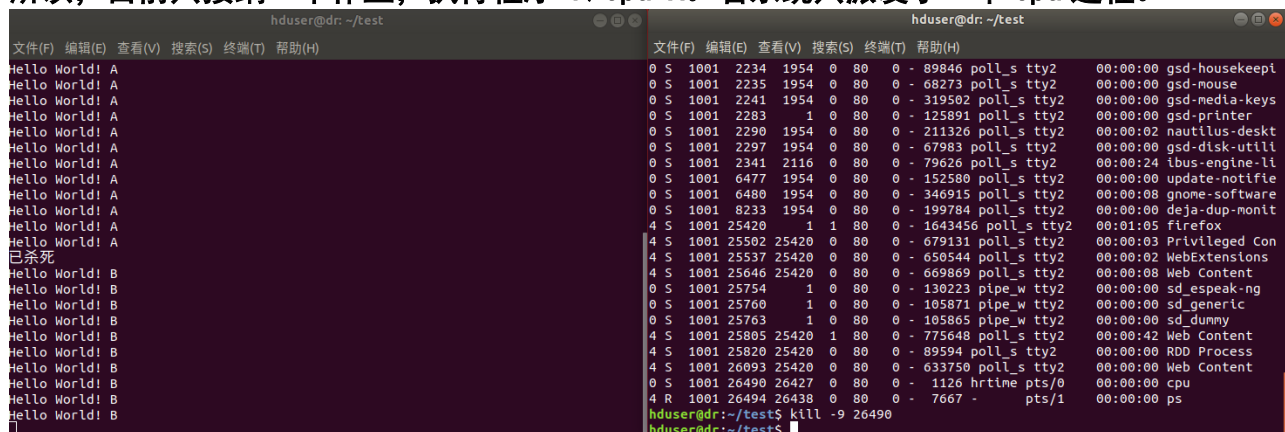


图 3

杀死 cpu A 进程后，接纳 ./cpu B 作业，创建一个新的 cpu 进程输出字符串
“Hello World! B”。

三、kill 命令

kill -9 2189；用 9#信号杀死 2189#进程。ctrl+c，用 2#信号杀死当前终端的前台进程。

输入命令行 ./cpu A； ./cpu B； ./cpu C； ./cpu D

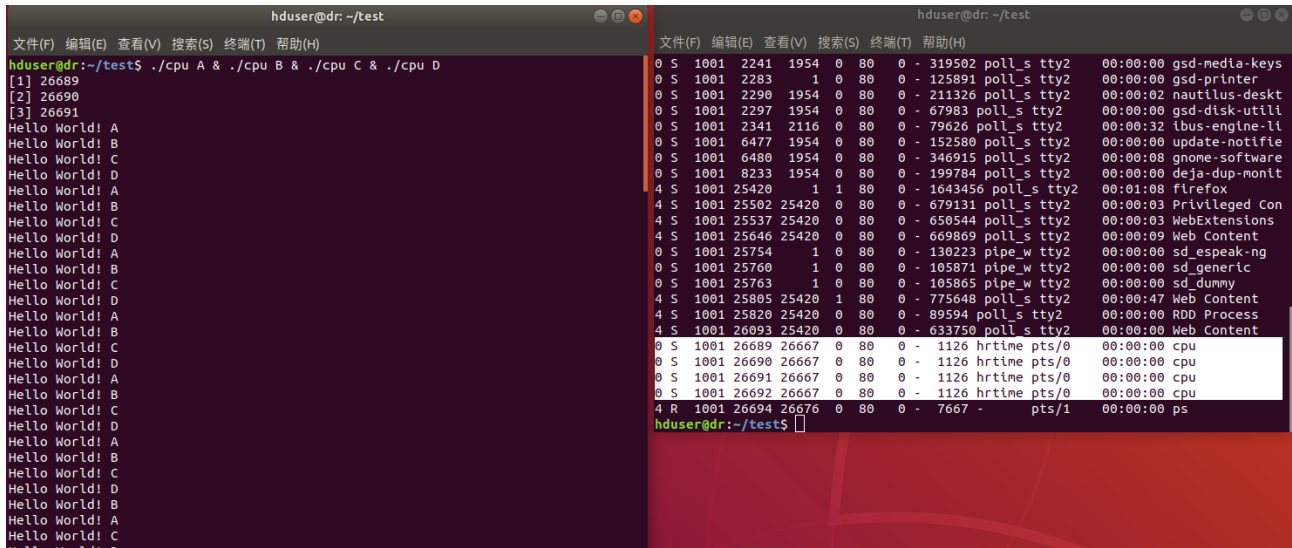
- 当前终端输入 ctrl+c，杀死前台作业 cpu A 后，shell 不会尝试启动后 3 个作业。
- 另启一个终端，kill -9 2189。杀死前端进程后，shell 会启动下一个作业 cpu B。
- 如果，我们在另一个终端，kill -2 2189。会看到 ctrl+c 一样的效果。

系统不同的行为，是因为 shell 使用不同的分支处理因收到 2#或 9#信号而终止的子进程。

四、“&”是后台作业的描述符。 所以，可以用来分割命令行中的多个作业。

一个终端可以提交很多后台作业。系统立即接纳后台作业，为它们创建足够多的进程。这里，4 个后台作业和负责执行它们的 4 个后台进程。

图 4（注，这里命令行 `./cpu D` 的后面少了一个 `&`。所以，这里只输出 3 个 PID 号）



怎样杀死后台进程

1、用 kill

2、关闭左侧终端也奏效。下图，关闭左侧终端后，`ps` 看不到 `cpu` 进程了。系统杀掉了这个终端的所有后台进程（操作前，系统有给我提示）。

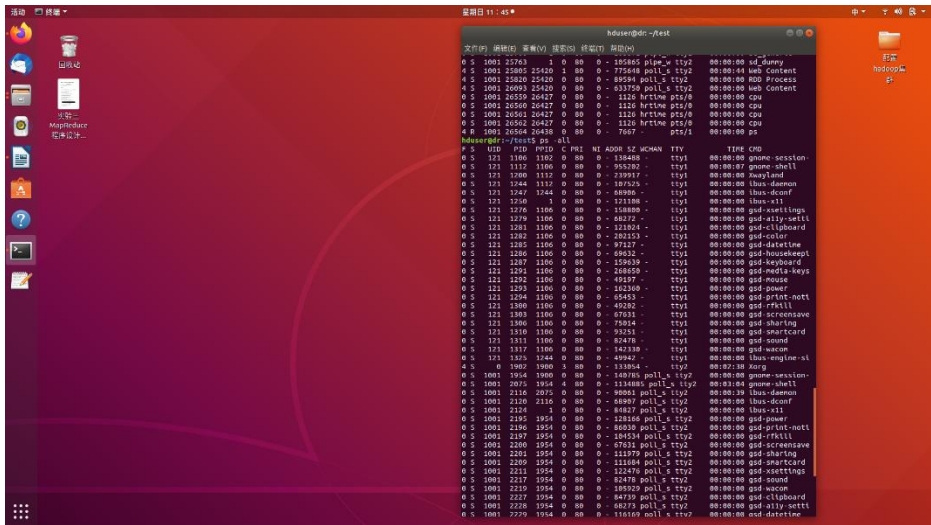


图 5

总结：

因为通用操作系统，作业的背景负载是无法预测、复杂多变的。所以，作业控制并不是一件容易的事情。